Documentation de conception

Erwan Didelot - Colin Dunand - Julia Mulder
Abel Prud'homme Lacroix - Baptiste Van Der Stracten

23 Janvier 2025

Contents

1 Analyse lexicale et syntaxique (étape A) 2

2 Vérifications contextuelles (étape B) 2
2.1 Architecture . . . 2
2.1.1 Description . . . 2
2.1.2 Étapes de la vérification contextuelle : . . . 3
2.1.3 Description des fonctionnalités implémentées . . . 3
2.2 Analyse contextuelle pour les programmes avec objet . . . 4

3 Génération de code (étape C) 6
3.1 Gestion des registres et des labels . . . 6
3.1.1 Gestion des registres . . . 6
3.1.2 Gestion des labels . . . 7
3.2 Sans Objet . . . 7
3.2.1 While . . . 7
3.2.2 IfThenElse . . . 7
3.2.3 Comparateur . . . 8
3.2.4 Opérateur Arithmétique . . . 8
3.3 Objet . . . 8
3.3.1 Spécifications supplémentaires . . . 8
3.3.2 Description des algorithmes et structures de données . . . 9
3.3.3 Exemple d'application . . . 9

1 Analyse lexicale et syntaxique (étape A)

Les fichiers concernés sont :

- `src/antlr4/fr/ensimag/deca/syntax/DecaLexer.g4`
- `src/antlr4/fr/ensimag/deca/syntax/DecaParser.g4`

Les fichiers ci-dessus implémentent l'analyseur lexical et syntaxique. Ils sont écrits en langage **Antlr** qui prend en entrée la grammaire attribuée fixée dans la spécification et la traduit en un programme capable de reconnaître les lexèmes définis dans `DecaLexer`, et transforme ce flux d'entrée au fur et à mesure en un arbre abstrait de syntaxes.

La liste des lexèmes manipulables par notre compilateur se trouve dans `DecaLexer`. D'autres lexèmes pourraient être ajoutés à cette liste (par exemple : `For`, `do...while`, etc.), si on veut ajouter d'autres règles à notre grammaire.

2 Vérifications contextuelles (étape B)

2.1 Architecture

2.1.1 Description

Les vérifications contextuelles s'appuient sur les répertoires suivants :

- `src/main/java/fr/ensimag/deca/context/*`
- `src/main/java/fr/ensimag/deca/tree/*`
- `src/main/java/fr/ensimag/deca/context/tools/symbolTable.java`

Cette phase vise à enrichir l'arbre abstrait généré par l'analyse syntaxique en y ajoutant des informations contextuelles et en détectant les erreurs de contexte potentielles. Cela est accompli en trois passes : lors de la passe 1, on vérifie le nom des classes et la hiérarchie de classes, lors de la passe 2, on vérifie les champs et la signature des méthodes des différentes classes et lors de la passe 3, on vérifie les blocs, les instructions, les expressions et les initialisations.

Pour les programmes sans objet, la troisième passe seule est donc suffisante pour valider les règles contextuelles.

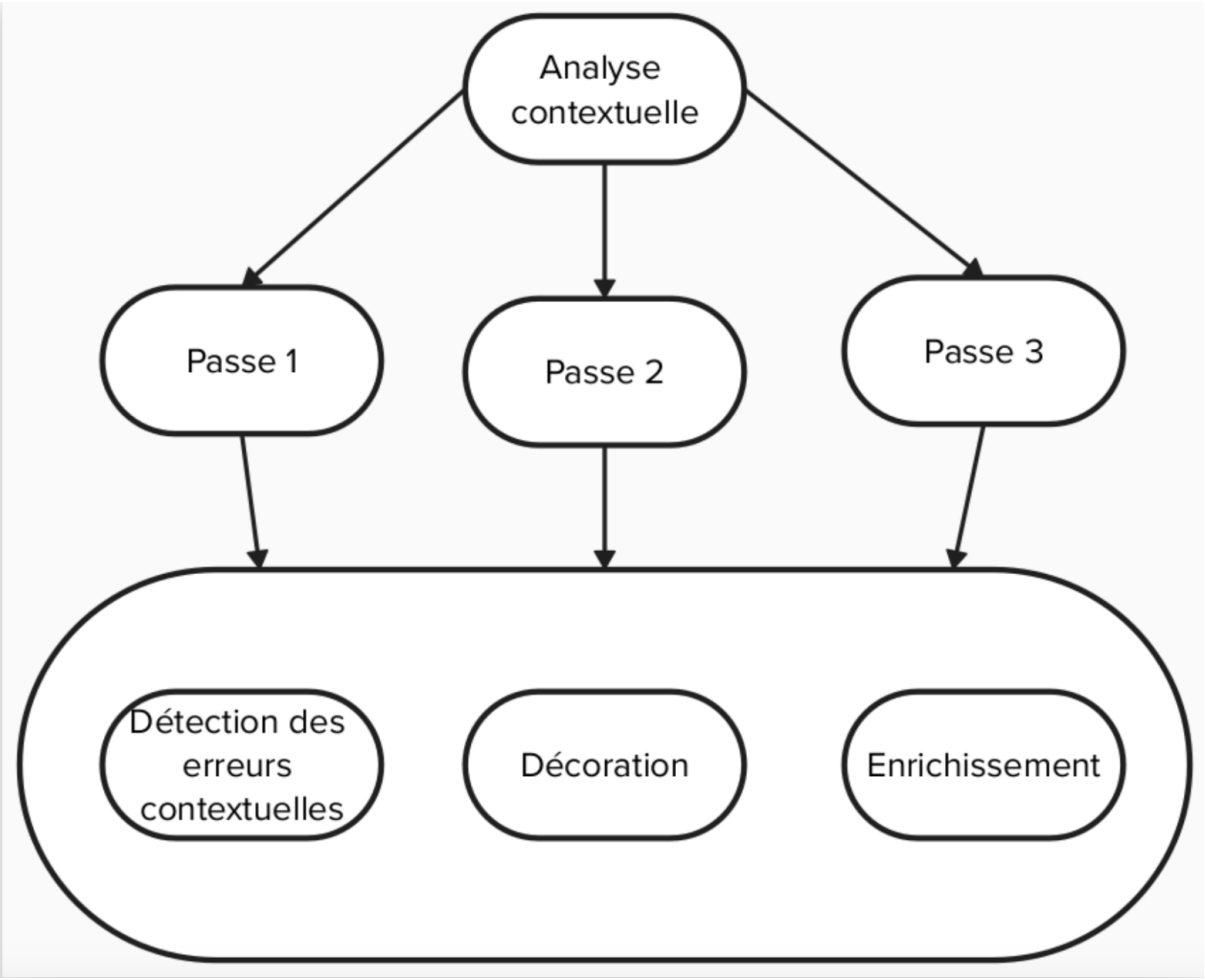


Figure 1: Etape B

2.1.2 Étapes de la vérification contextuelle :

- **Détection d'erreurs :**
  - Identifier les violations des règles de grammaire attribuées.
  - Un message d'erreur est levé si le flux d'entrée ne respecte pas ces règles.
  - Cela est réalisé grâce à l'utilisation des domaines attribués :
    - `Environment_type` (similaire à `Environment_exp`).
    - `Environment_type` enregistre les symboles et leurs définitions de types dans un dictionnaire. Les getters et setters facilitent la manipulation de ces données.
- **Décoration de l'arbre syntaxique :**
  - Les typages provenant des environnements (`Environment_type` et `Environment_exp`) sont ajoutés au fur et à mesure des trois passes.
  - L'arbre abstrait devient ainsi décoré et contextuellement enrichi.

2.1.3 Description des fonctionnalités implémentées

Fonctions Verify :

- Vérification contextuelle des expressions via les fonctions :
  - `verifyExpr`
  - `verifyRvalue`

  - `verifyCondition`

- Ces fonctions utilisent les environnements `Environment_exp` et `Environment_type` (champ du paramètre `compiler` de type `DecacCompiler`).

Classe EnvironmentType :

- Contient un dictionnaire associant chaque `Symbol` (défini dans `tools/symbolTable.java`) à une `TypeDefinition` ou à une `ClassDefinition` (pour les classes).
- A noter que suite à de nombreux problèmes de définition sur l'utilisation des environnements, nous avons décidé de transformer tous les symboles en `String` avant de les ajouter dans un environnement.
- Fournit des méthodes pour :
  - Initialiser l'environnement prédéfini.
  - Récupérer des définitions de types déjà existants.

Détection des erreurs :

- Les erreurs contextuelles sont identifiées lors des passes grâce à des tests directement implémentés dans les nombreux `verifys`.
- Si une incompatibilité est détectée, un message d'erreur est levé, indiquant la position de l'erreur et le type incompatible via des `throw new ContextualError`.

Décoration et enrichissement :

- Une fois les vérifications terminées (à la fin de `verifyProgram`), les types ou définitions nécessaires sont initialisés et ajoutés aux nœuds correspondants.
- Cela se produit à la fin du traitement de chaque nœud (dans les fonctions `verifyExpr`, `verifyRvalue` ou `verifyCondition`).

2.2 Analyse contextuelle pour les programmes avec objet

Les programmes avec objet sont des fichiers sources contenant des classes définies selon les spécifications du cahier des charges. Bien que l'architecture décrite précédemment soit similaire, certaines classes spécifiques ont été ajoutées ou modifiées pour intégrer les caractéristiques de la programmation orientée objet (encapsulation, héritage, etc.).

Classes principales et leurs rôles

- **DeclClass :**
  - Hérité de `AbstractDeclClass` (elle-même héritant de `tree`).
  - Implémente les passes 1, 2 et partiellement 3 via les méthodes :
    * `verifyClass`
    * `verifyClassMembers`
    * `verifyClassBody`
- **DeclField :**
  - Hérite de `AbstractDeclField` (héritant de `tree`).
  - Vérifie les champs de la classe :
    * Existence de champs du même nom dans une classe mère.
    * Attribution des index appropriés.
    * Ajoute chaque champ dans l'environnement local.
- **DeclMethod :**
  - Hérite de `AbstractDeclMethod` (héritant de `tree`).

  - Vérifie les méthodes en effectuant les passes 2 et 3 via des fonctions dédiées :
    * `verifyMethodSignature` : Implémente la deuxième passe, vérifiant la bonne initialisation de la méthode et son ajout dans l'environnement des méthodes définie dans `Environment_exp`.
    * `verifyMethodBody` : Implémente la troisième passe, vérifiant les instructions et les déclarations dans le corps de chaque méthode.

- **MethodCall :**
  - Hérite de `AbstractExpr`.
  - Gère les appels de méthodes en vérifiant :
    * L'existence de la méthode dans l'environnement des méthodes.
    * La compatibilité des signatures entre la méthode appelante et la méthode appelée.

- **DeclParam :**
  - Hérite de `AbstractDeclParam` (héritant de `tree`).
  - Ajoute chaque paramètre dans l'environnement local après avoir vérifié :
    * L'unicité des noms dans le contexte (erreur levée en cas de doublon).

- **ThisLiteral :**
  - Hérite de `AbstractExpr`.
  - Désigne l'objet courant de la classe dans laquelle il est utilisé.
  - Vérifie que `this` n'est pas appelé dans le `main`.

- **ListDeclClass :**
  - Hérite de `TreeList<AbstractDeclClass>`.
  - Lance la vérification pour chaque classe déclarée dans le fichier source.

- **ListDeclField :**
  - Hérite de `TreeList<AbstractDeclField>`.
  - Lance la vérification pour chaque champ déclaré dans le fichier source.

- **ListDeclParam :**
  - Hérite de `TreeList<AbstractDeclParam>`.
  - Lance la vérification pour chaque paramètre déclaré dans le fichier source.

- **ListDeclMethod :**
  - Hérite de `TreeList<AbstractDeclMethod>`.
  - Lance la vérification pour chaque méthode déclarée dans le fichier source.

- **New :**
  - Hérite de `AbstractExpr`.
  - Vérifie l'instanciation des classes dans le fichier source.

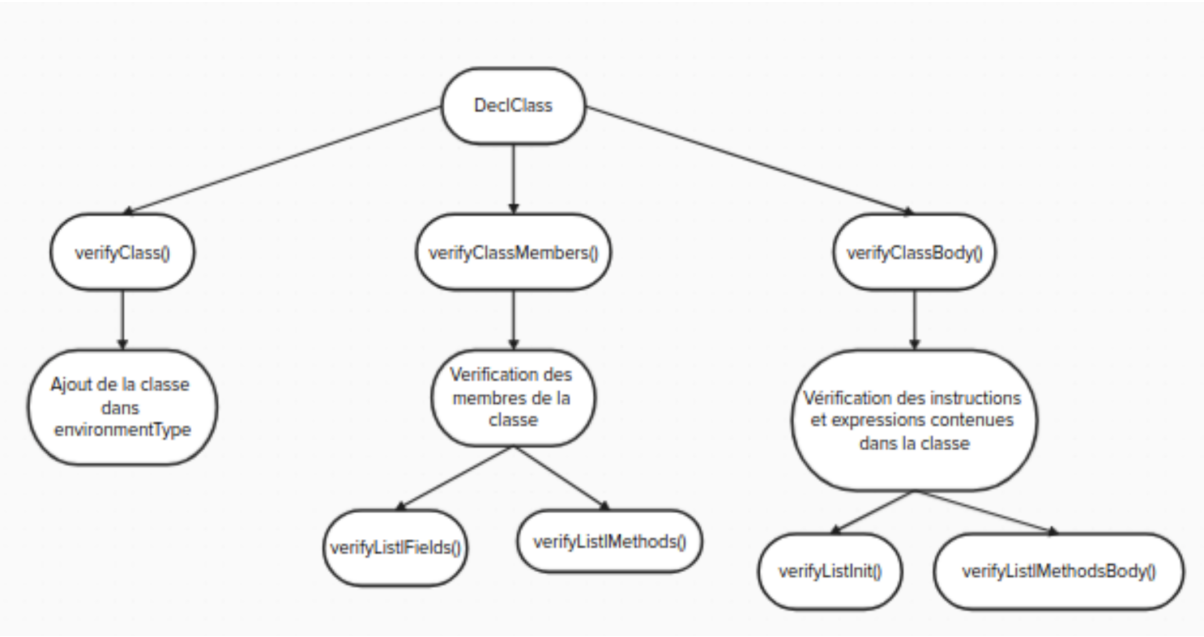


Figure 2: Fonctionnement de la déclaration d'une classe

3 Génération de code (étape C)

3.1 Gestion des registres et des labels

3.1.1 Gestion des registres

Pour gérer les registres, nous avons créé un fichier `RegisterManag.java` dans le package `/codegen`.

`RegisterManag` est une classe possédant deux champs :

- `availableRegisters`, une LinkedList qui est initialisée au départ de la génération de code et qui contient des registres de type `GPRegister` (le nombre de registres libres stockés dans cette liste dépend de si l'option -r X est utilisée avec `decac` : si c'est le cas la liste contiendra les registres R2 jusqu'à R[X] avec $4 \leq X \leq 16$, sinon elle contiendra tous les registres de R2 à R15). Les registres R0 et R1 sont particuliers et ne sont donc pas stockés dans cette liste.
- `lastUsedRegister`, un registre qui est initialisé à `null` et qui servira à garder en mémoire le dernier registre utilisé.

Cette classe possède plusieurs méthodes qui permettent d'agir sur ces deux champs.

La fonction `allocateRegister` qui renvoie un registre, récupère le premier registre disponible de la liste `availableRegisters`, l'enlève de la liste car on va l'utiliser donc il n'est plus disponible et le renvoie. Cette fonction change aussi la variable `lastUsedRegister` en lui attribuant le registre libéré.

La fonction `freeLastUsedRegister` libère le dernier registre utilisé, elle remet donc le `lastUsedRegister` dans la liste `availableRegisters` à la première place et une fois cela fait redonne à `lastUsedRegister` la valeur `null`.

La fonction `freeRegister(GPRegister reg)` libère le registre qui lui est donné en argument en le remettant à la première place dans la liste `availableRegisters`.

La manière dont est implémentée la génération de code fait que les registres vont être remis dans l'ordre initial dans lequel ils étaient stockés (donc en commençant par R2).

Cette classe contient aussi des getters et des setters pour modifier et récupérer `availableRegisters` et `lastUsedRegister`.

3.1.2 Gestion des labels

Un des problèmes qui s'est posé lors de l'implémentation de la génération de code a été la gestion des labels. Il faut un label différent pour chaque boucle while, chaque branche if then else,... Pour cela, nous avons créé un fichier `./src/main/java/fr/ensimag/deca/codegen/RegisterManag.java` dans le package `/codegen`.

Ce fichier contient une classe `LabelManager` avec comme attributs `counterWhile`, `counterIfthenElse`, `counterCmp`, `counterAnd` et `counterOr`. Ces attributs sont des compteurs qui permettent de créer à chaque nouveau While ou If then else, à chaque nouvelle comparaison et à chaque nouveau "ET" ou "OU" un nouveau label numéroté ce qui permet de les différencier. Les labels sont créés de la manière suivante (par exemple pour un If then else) :

```
int numberIf = compiler.getLabelManager().counterIfThenElse();
Label endLabel = new Label("if_end" + numberIf);
```

3.2 Sans Objet

A chaque fois que l'on entre dans une nouvelle branche, on prend le dernier registre utilisé grâce à la classe de gestion des registres.

3.2.1 While

Pour générer les While, nous avons utilisé la structure suivante :

```
    BRA while_cond1
while_start1:
    "body_code"
while_cond1:
    "condition_code"
    BNE while_start1
    "suite_du_code"
    HALT
```

3.2.2 IfThenElse

Pour générer les IfThenElse, nous avons utilisé la structure suivante pour un if - else :

```
    "condition1_code"
    BNE if_cond1
    "body_else_code"
if_cond1:
    "body_if_code"
    BRA if_end1
if_end1:
    "suite_du_code"
    HALT
```

Nous avons utilisé la structure suivante pour un if - else if - else:

```
    "condition1_code"
    BNE if_cond1
    "condition2_code"
    BNE if_cond2
    "body_else_code"
if_cond1:
    "body_if1_code"
    BRA if_end1
if_cond2:
    "body_if2_code"
    BRA if_end1
```

```
if_end1:
    "suite_du_code"
    HALT
```

3.2.3 Comparateur

Pour générer les comparateurs, nous avons utilisé la structure suivante :

```
    "left_operand_code"
    "right_operand_code"
    CMP R3, R2 // comparaison des deux derniers registres utilisés
    BGT cmp_label1 // utilisation de la bonne instruction
    LOAD #0, R2
    BRA cmp_end1
cmp_label1:
    LOAD #1, R2
cmp_end1:
    "suite_du_code"
```

3.2.4 Opérateur Arithmétique

Pour générer les opérateurs arithmétiques, nous avons utilisé la structure suivante :

```
    "left_operand_code"
    "right_operand_code"
    ADD R3, R2
    // utilisation des deux derniers registres utilisés avec la bonne instruction
```

Dans le cas où les calculs s'enchaînent, la génération descend à gauche dans l'arbre avant de partir à droite afin d'utiliser moins de registres.

3.3 Objet

Informations importantes : à cause d'un problème de merge, nous n'avons pas pu push à temps la passe 1 de la partie C mais nous avons quand même réussi à générer du code pour cette passe.

On trouve les classes suivantes dans cette partie :

- **Classes principales et leurs dépendances** : les mêmes que dans l'étape B.
- **Classes Definition et EnvironmentExp** : elles fournissent une structure de données pour gérer les environnements de classes et les relations d'héritage et elles stockent les informations nécessaires à la génération des tables des méthodes.

3.3.1 Spécifications supplémentaires

- **Génération des tables des méthodes :**
  - Les méthodes héritées sont copiées dans la table des méthodes de la sous-classe.
  - Les méthodes redéfinies remplacent leurs versions héritées tout en conservant leur index dans la table.
- **Gestion de l'héritage :**
  - Lorsqu'une classe hérite d'une autre, un pointeur vers la table des méthodes de la superclasse est stocké.
  - Chaque méthode est associée à une étiquette (`Label`) utilisée pour générer le code machine.
- **Initialisation de la classe Object :**
  - Une méthode spéciale, `initObjectMethods`, initialise la classe `Object` avec une méthode par défaut `equals`.

- Cette méthode sert de base pour toutes les autres classes.

- **Génération de code pour les déclarations de variables :**
  - Les variables sont initialisées avec des valeurs par défaut (comme 0) si il n'y a pas d'assignation.
  - Les registres sont gérés dynamiquement pour optimiser l'utilisation des ressources.

3.3.2 Description des algorithmes et structures de données

- **Algorithme pour les tables des méthodes :**
  - On identifie la superclasse,
  - on copie les entrées de la table des méthodes de la superclasse,
  - on ajoute ou on remplace les méthodes locales en respectant l'ordre des indices.

- **Structure des environnements :**
  - Chaque classe possède un environnement (`EnvironmentExp`) pour ses membres.
  - Les environnements sont liés entre eux pour refléter l'héritage.

- **Algorithme d'allocation des registres :**
  - Les registres disponibles sont gérés dans une liste (`availableRegisters`).
  - Les registres sont alloués temporairement et libérés après utilisation.

3.3.3 Exemple d'application

Pour le code suivant :

```
class A {
    void m();
    void p();
}

class B extends A {
    void p();
    void q();
}
```

- **Génération des tables des méthodes :**
  - La classe `Object` génère une table avec la méthode `equals`.
  - La classe `A` a hérité de `Object` et ajoute les méthodes `m` et `p`.
  - La classe `B` hérite de `A`, redéfinit `p` et ajoute `q`.

- **Justifications :**
  - L'utilisation des étiquettes (`Label`) permet une gestion efficace des appels de méthodes.
  - Les pointeurs vers les tables des méthodes des superclasses facilitent la gestion de l'héritage.